

PROJECT REPORT OF CO-OP Project at Industry

(Module-2) ON

**Performance Comparison of REST and GraphQL APIs Under
Variable Load Conditions**

**submitted in partial fulfilment of the requirements for the award of degree
of**

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

**Submitted by:
Supervised By:**

**Gopal Krishna Shrivastava(2210990327)
Swpanil Dhamija (2210990881)
Tritiya (2210990914)**

Dr. Anshu Singla

**Department of Computer
Science and Engineering,
Chitkara University, Punjab**



**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page No.
	Acknowledgement	3
	List of Figures and Tables	4
	Abstract	5
1	Introduction	2
2	Methodology	3
3	Tools and Technologies	5
4	Implementation	6
5	Major Findings/Outcomes/Results	7
6	Conclusion and Future Scope	9
	References	11
	Appendices	12

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**Performance Comparison of REST and GraphQL APIs Under Variable Load Conditions: A Comprehensive Empirical Study with Security and Scalability Analysis**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Anshu Singla**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Pune

Date: 18/5/2026

Gopal Krishna Shrivastava(2210990146)

Swapnil Dhamija(2210990881)

Tritiya (2210990914)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Anshu Singla

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We express our sincere gratitude to Chitkara University Institute of Engineering and Technology for providing the environment and resources to explore the theoretical foundations and practical applications of this research. Special thanks to the researchers and industry professionals whose foundational work on API architectures inspired this comprehensive empirical study.

Abstract

Application Programming Interfaces (APIs) form the backbone of modern web and mobile systems, enabling communication between client applications and backend services. Among contemporary API paradigms, Representational State Transfer (REST) and GraphQL have emerged as dominant architectural approaches. REST emphasizes resource-oriented endpoints and standardized HTTP methods, whereas GraphQL introduces a query-based model that allows clients to request precisely the data they require. While both paradigms are widely adopted, performance-related trade-offs between them remain context-dependent and are often evaluated under idealized conditions rather than controlled experimental workloads.

This study presents a systematic empirical comparison of REST and GraphQL APIs under variable load conditions to evaluate their performance characteristics, scalability behavior, and resource efficiency. Two functionally equivalent backend services are implemented using a consistent technology stack to eliminate confounding variables. Both APIs interact with identical data models and databases, ensuring parity in business logic and persistence layers. The experimental setup deploys both implementations in a controlled environment where hardware constraints, concurrency levels, and request distributions are explicitly defined.

Performance evaluation is conducted using industry-standard load testing tools such as Apache JMeter and k6. The systems are subjected to incremental user loads ranging from low concurrency (100 users) to high concurrency (1000+ users). Key performance metrics include mean and 95th percentile response times, throughput (requests per second), payload size efficiency, CPU utilization, memory consumption, and network bandwidth usage. Both steady-state and burst traffic scenarios are analyzed to observe degradation patterns and system stability under stress.

The study also investigates query complexity effects in GraphQL, particularly how deeply nested or over-fetching queries impact execution time and server resource utilization compared to REST's fixed endpoint structure. Additionally, caching behavior and response predictability are examined to assess their influence on scalability and performance consistency.

Preliminary expectations suggest that REST may demonstrate lower computational overhead under uniform request patterns due to its simpler request-processing model and cache-friendly architecture. Conversely, GraphQL may exhibit superior payload efficiency in scenarios involving selective data retrieval, potentially reducing network overhead but increasing server-side query processing complexity. The findings aim to clarify whether performance differences are intrinsic to the

architectural paradigm or primarily influenced by implementation strategies and workload characteristics.

The contribution of this research lies in providing a controlled, data-driven evaluation of REST and GraphQL APIs under realistic and measurable load conditions. Rather than advocating for a universally superior approach, the study seeks to contextualize architectural selection within workload requirements, scalability expectations, and infrastructure limitations. The results are intended to inform backend developers, system architects, and organizations seeking evidence-based guidance when selecting API technologies for performance-sensitive applications.

1. Introduction

The fundamental communication layer underlying almost every modern information technology infrastructure is the application programming interface (API). Whether a mobile client retrieving personalized content or microservices communicating in a chain, there is always an API transaction. The decision of a chosen API design strategy directly sets the bounds of the scalability of an entire software system.

Representational State Transfer (REST) has been the most prevalent API architectural style for over 10 years, utilizing stateless request verbs and resource-oriented URI structures over HTTP. However, contemporary environments contain data constraints on deployments that REST's endpoint-oriented, server-prescribed delivery requirements are not very effective at managing. Clients frequently receive superfluous data (over-fetching) or must make sequential requests to gather data (under-fetching).

To address these inefficiencies, Facebook built GraphQL in 2012, bringing all interactions together into one endpoint and presenting a query language that allows clients to explicitly specify the required data. While GraphQL solves payload issues, it introduces computational burdens associated with heavy loads and unique security consequences due to its single entry point. This paper tackles these aspects with a rigorously conducted experimental comparison and expands the scope by adding gRPC, a paradigm that has recently demonstrated higher throughput than both REST and GraphQL.

2. Methodology

The experimental design and parity controls were structured so that the API paradigm being evaluated was the only independent variable. All remaining parameters, including the language runtime, object-relational mapper, data structure, database, and hardware setup, were fixed.

The data schema represented a typical e-commerce domain comprising Users, Orders, Products, and Reviews. Workloads were evaluated across three levels of query complexity: Tier 1 (Shallow), Tier 2 (Relational), and Tier 3 (Nested). Primary performance indicators measured included mean latency, 95th-percentile latency, CPU utilization, and resident set size (RSS) memory consumption. Performance evaluation involved structured load generation spanning from an initial baseline phase at 100 virtual users (VU) to a stress phase scaling linearly up to 2,000+ VU over a 5-minute ramp interval.

3. Tools and Technologies

- Runtime: Node.js v20 LTS was used as the runtime.
- API Frameworks: Express.js v4 was utilized for the REST implementation, while Apollo Server v4 was used for GraphQL.
- Database and ORM: PostgreSQL 15 served as the shared database, accessed via Prisma ORM v5, which was optimized to generate the same SQL execution plan for both implementations.
- Cloud Infrastructure: AWS EC2 t3.medium instances (2 vCPUs, 4GB RAM) were used to host the services.
- Load Generation: A dedicated AWS EC2 t3.large instance was utilized for load generation to remove co-location interference, employing Apache JMeter and Grafana k6.
- Profiling Tools: Linux top and the `node --prof` profiling toolchain were used to capture CPU and memory metrics.

4. Implementation

Testing execution was broken down by specific operation tiers:

Tier 1 required 1 REST GET operation to fetch a user by ID.

Tier 2 required 2 REST GET operations to fetch a user and their orders.

Tier 3 required 3 REST GET operations to fetch a user, orders, products, and reviews.

During execution, flame-graph analysis was conducted during the 2,000-VU test phase to isolate computational bottlenecks, monitoring memory consumption sampled every 10 seconds.

5. Major Findings/Outcomes/Output/Results

The experimental analysis revealed distinct operational and security characteristics for each API paradigm:

Throughput & Scalability: Under stress conditions, REST throughput scaled near-linearly to a peak of 12,500 RPS at 2,000 VU, maintaining a 0% error rate. GraphQL reached a maximum throughput of 7,200 RPS at 1,500 VU before declining to 6,100 RPS at 2,000 VU, exhibiting a 4% HTTP 500 error rate.

CPU & Memory Utilization: At 2,000 VU, GraphQL hit near-saturation at 94% CPU utilization, while REST retained 38% headroom. Flame-graph analysis showed that 28% of GraphQL CPU cycles were expended exclusively on query lexing, AST construction, and schema validation. Peak memory (RSS) for Apollo Server reached 142 MB compared to 88 MB for the Express REST service.

Payload Efficiency: GraphQL produced substantial payload reductions that compounded with nesting depth, achieving a 66.6% reduction for Tier 1, 82.1% for Tier 2, and 88.1% for Tier 3. Under peak burst traffic (2,000 VU, Tier 3), payload reduction peaked at 94.0%.

Security Posture: REST presented a narrow default attack surface with high security maturity, natively supporting HTTP caching which acts as a DoS buffer. GraphQL exhibited high risks for schema exposure and DoS via query complexity ("complexity bombs"). It natively provides a schema query allowing introspection abuse and requires deliberate rate-limit enforcements per operation rather than per request.

gRPC Extended Comparison: In three-way benchmarks, gRPC dominated throughput and CPU efficiency due to its binary serialisation format (Protobuf), though it lacks native browser support without a gateway.

6.Conclusion and Future Scope

The study demonstrates that the selection of an API paradigm depends on the specific characteristics and constraints of the system. REST shows a higher suitability profile when throughput capacity, operational stability, and high-volume uniform CRUD operations are principal design priorities. GraphQL remains the technically sound option for bandwidth-constrained deployment targets and data-heavy tasks, provided engineering teams invest in batching utilities like DataLoader and strict

security hardening . gRPC is recommended for internal service-to-service communication scenarios where low latency and data transfer efficiency are paramount. Hybrid architectural patterns, such as combining a REST or gRPC internal backbone with a GraphQL Backend-for-Frontend, represent a pragmatic approach to capturing complementary strengths.

Future research directions include studying the impact of HTTP/3 (QUIC) on the latency of REST in high-packet-loss mobile network conditions. Furthermore, research should investigate whether Automatic Persisted Queries (APQs) can close the structural caching gap at scale with GraphQL, and analyze the performance of triple-paradigm Backend-for-Frontend architectures operating simultaneously.

Keywords: REST, GraphQL, gRPC, API performance, load testing, throughput, latency, payload efficiency, N+1 query, HTTP caching, security, microservices.

References

- [1] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine. 2000.
- [2] M. Mikuła, M. Kowal, and P. Rzeźnik, "Performance Analysis of REST and GraphQL APIs in Web Applications," J. Comput. Sci. Eng., vol. 15, no. 3, pp. 112-124, 2022.
- [3] A. Saleh and W. Nawara, "Evaluating GraphQL and REST API Services Performance in a Massive and Intensive Accessible Information System," MDPI Computers, vol. 10, no. 11, p. 138, Oct. 2021.
- [4] M. Niswar, R. A. Safruddin, A. Bustamin, and I. Aswad, "Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC," Int. J. Electron. Telecommun., vol. 70, no. 2, pp. 429- 436, Jun. 2024.
- [5] G. Mavroudeas and S. Makris, "A Study on REST and GraphQL APIs," in Proc. IEEE Int. Conf. Comput. Inf. Technol., 2019.
- [6] GraphQL Foundation, "GraphQL Specification," Jun. 2018.
- [7] Postman Inc., "2023 State of the API Report," 2023.
- [8] Postman Inc., "API Survey: GraphQL Ranks Third in API Architecture Adoption," 2023.
- [9] L. Byron, "DataLoader Batching and Caching for GraphQL," Facebook Engineering, 2016.
- [10] OWASP Foundation, "GraphQL Cheat Sheet," OWASP Cheat Sheet Series, 2024.

[11] Imperva Security Research. "GraphQL Vulnerabilities and Common Attacks Seen in the Wild," Mar. 2025.

[12] PortSwigger Web Security Academy, "GraphQL API Vulnerabilities," 2023.

[13] HackerOne, "How a GraphQL Bug Resulted in Authentication Bypass," 2023.

[14] Wiz Security, "GraphQL API Security Risks Every Developer Should Know About," Jan. 2026.